

№11 ООП

План

- DDD
 - 1. Модель
 - 2. Структурные элементы
 - 3. Архитектура
 - 4. Тестирование
 - 5. Рефакторинг
 - Причины рефакторинга
 - 6. Контексты

DDD

Паттерн - решение задачи в контексте.

DDD - *Domain Driven Design. Предметно Ориентированное Проектирование*. По сути это сборник правил проектирования.

DDD вообще состоит из четырёх элементов, но он дал шесть.

1. Модель

Модель - формирование модели в терминах предметной области. В этом процессе *обязательно* должен участвовать заказчик. Этот этап *поможет наладить понимание* предметной области и исключит абсурдные архитектурные решения со стороны заказчика.

2. Структурные элементы

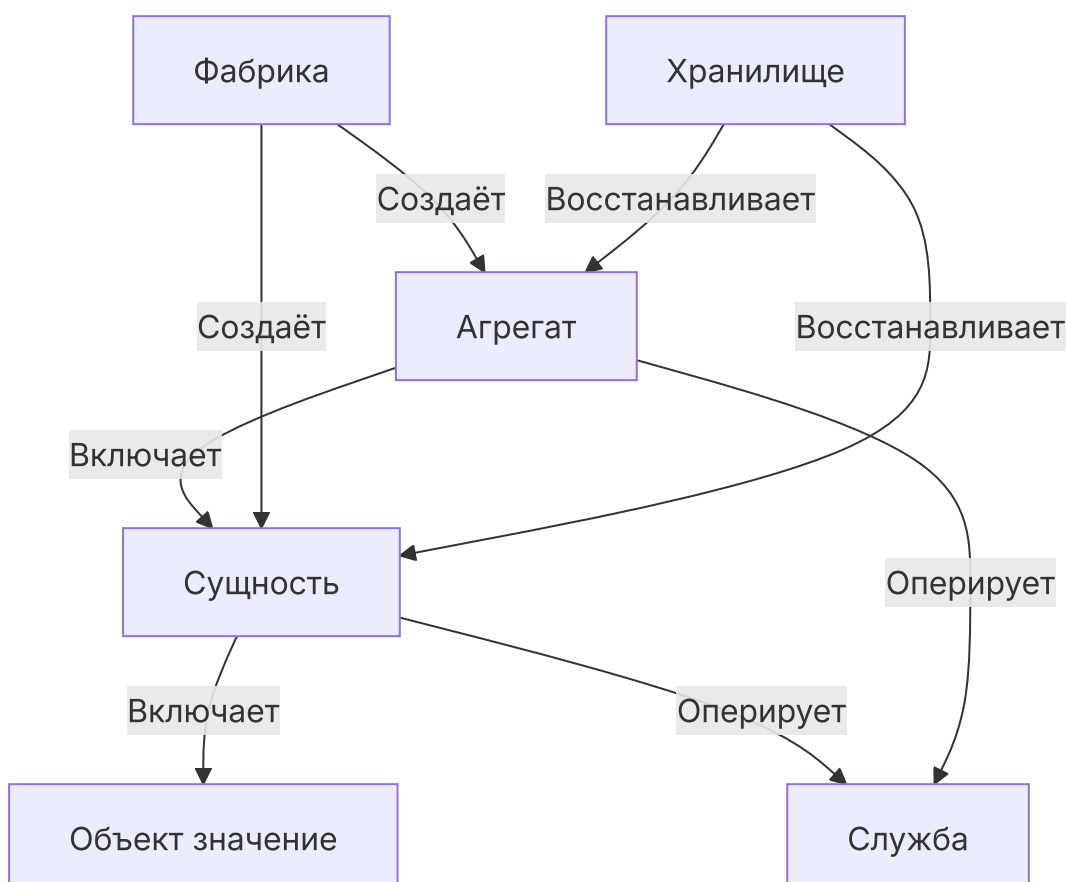
Структурные элементы - на этом этапе нам надо *разобраться с типами элементов* внутри модели. Объекты бывают двух типов:

1. **Объекты модели.**

1. *Объект значение* - он же `struct`, объект, который *не содержит* методов. *Контейнер. Нет идентичности*, они не уникальны. Как правило *входит в состав сущностей*.
2. *Сущность* - *полноценный объект* с поведением и состоянием. *Есть идентичность*, нам важно с кем мы работаем.
3. *Агрегат* - включает в себя *набор сущностей с выделением корневой сущности*, для реализации общего функционала. По сути это набор сущностей, связанных через корневую сущность, *для выполнения одной задачи*. Например, как ветви `systemd`.

2. Другие типы объектов.

1. *Службы* - они же `utils`. Это объекты *без собственного функционального состояния*. Определены только операции. *Со службами работают только сущности и агрегаты*.
2. *Фабрика* - фабрика *создаёт сущности и агрегаты*.
3. *Хранилище* - идентичный интерфейс с фабрикой. *Сохраняет и восстанавливает функции агрегата*.



3. Архитектура

В DDD принято использовать *многоуровневую слоевую архитектуру*.

Есть четыре уровня:

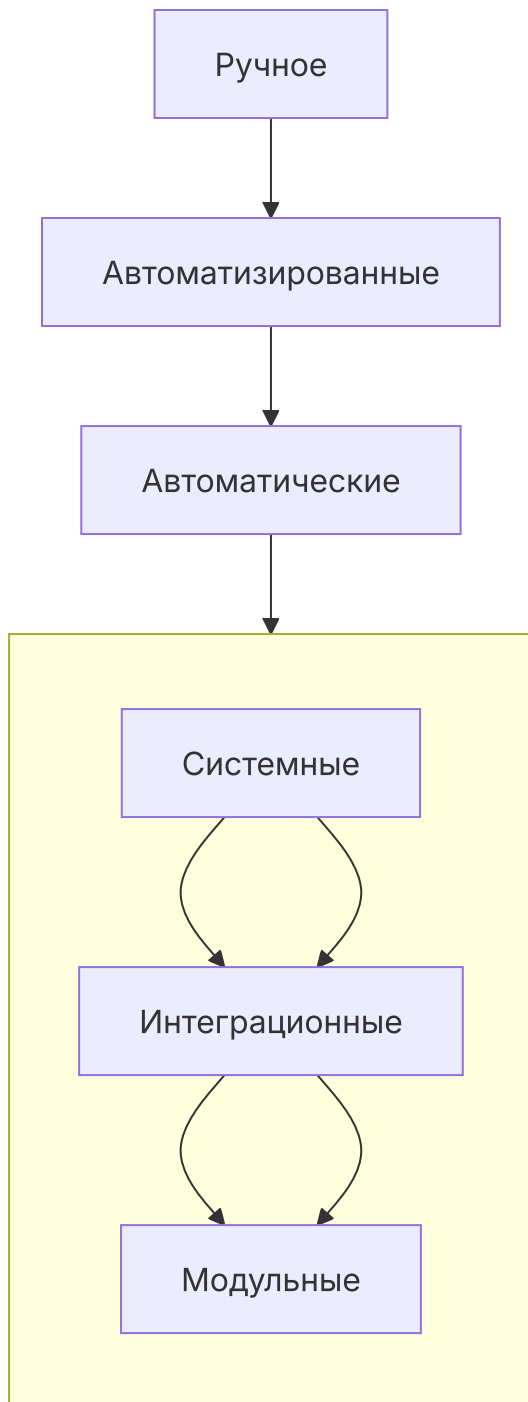
1. **UI** - GUI. Не в коем случае *не описывать SmartUI*.
2. **Операционный** - сценарии. Этот уровень направлен на *формирование сценариев* пользовательских историй.
3. **Доменный** - объекты, бизнес-логика. На этом уровне нужно *убедиться, что инварианты* (варианты порядка вызова методов), *не могут положить логику*.
4. **Инфраструктурный** - фреймворк, ОС, драйвер.

Причём *вызовы* между этими уровнями идут *строго сверху вниз*.

4. Тестирование

Тестирование - процесс, в котором определяется, что *система не содержит определённого вида дефектов*.

Есть две пирамиды тестирования:



Вторая пирамида является структурой автоматического тестирования.

По сравнению с архитектурой:

1. **UI** - ручное.
2. **Операционный** - модульный + интеграционный.
3. **Доменный** - модульный.
4. **Инфраструктурный** - не тестируемый.

Системное тестирование проходит по всей архитектуре. Как оно проходит:

1. Создаётся `Dummy` - пустой объект.
2. Ему выдаются `Stub` - заглушки полей.
3. Создаётся `Spy` для слежки за состоянием.
4. Создаётся `MOC` - истинная разметка тестирования.

5. Рефакторинг

Рефакторинг - процесс изменения структуры кода без изменения его функциональности. *Цель* рефакторинга - *повышение читабельности*.

Механизмы рефакторинга подробно описаны в книге Мартина Фаулера - Рефакторинг.

Рефакторинг в DDD нужен для *уточнения модели* предметной области. Сущности становятся более выраженными. Это называется *углубляющий рефакторинг*.

Причины рефакторинга

- **Дублирующий код.**
 - Дублирование *алгоритмов* - кодовая база раздувается.
 - Дублирование *сущностей* - модель становится нечёткой.
 - Дублирование *данных* - проблемы синхронизации.
- **Стрельба дробью** - логика размазана по разным модулям и изменения требуют вносить изменения в большую часть программы.
- **Параллельные иерархии наследования** - реализация нескольких классов для разных иерархий.
- **Завистливые классы** - подпрограммы лезут не в свои объекты.
- **Наличие комментариев** - если хочется написать комментарий, то время писать подпрограмму. Уместны только `docString`. Для информации есть документация.

- **Одержимость примитивами** - логика становится слишком мелкой.
- **Группы данных** - группы данных нужно объединять в объект.
- **Цепочка сообщений** - `a.getC().getD().detM().params()`. Допустимо не более трёх. Для упрощения пишем делегата, отдельный класс или метод.
- **Наличие делегата** - делегат есть, но он не решает несуществующую проблему. Убиваем делегата.
- **Длинные логические выражения** - `(c>3) || (d==1)&&(g==st)`. Сделать подпрограмму `isSomething`.
- **Незначащий класс** - класс который ничего не делает.
- **Слабая связь части параметров** - параметр в классе, который не используется в большинстве методов. Нужно выносить в отдельный класс.
- **Отказ от наследства** - наследование не реализует ни структуру ни поведение.
- **Магические цифры** - константы без констант. Решается константами.

6. Контексты

Контекст - это абстракция, которая позволяет *ввести две и больше модели* для различных предметных областей. Инкапсуляция участка модели предметной области.

Стратегии объединения контекстов:

- *Конформизм* - назначаем `master` и `slave` контексты.
- *Не пересекающиеся контексты* - объединение не требуется.
- *Введение антикоррупционного уровня* - прослойка между контекстами.
- *Введение общего ядра* - вводится третий контекст - пересекающиеся сущности.

- *Заказчик - поставщик* - один контекст является заказчиком, другой контекст предоставляет реализацию.